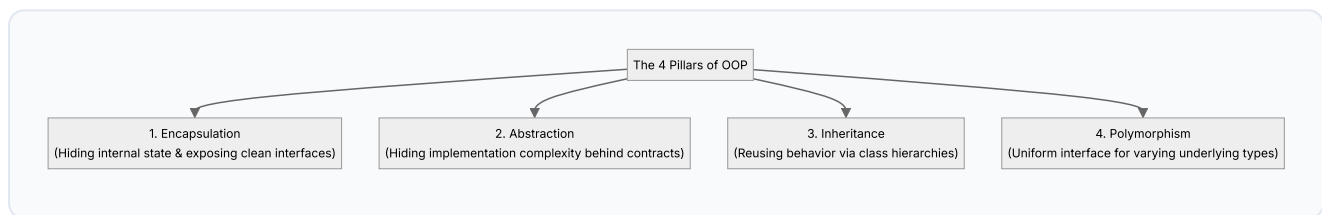


LLD Chapter 1: OOP Fundamentals & SOLID Principles in Modern Python

Core Learning Objective: Master Object-Oriented Design principles from absolute first principles up to senior enterprise architecture. Understand encapsulation, abstraction, inheritance, polymorphism, and master all 5 SOLID principles with concrete anti-patterns and production refactors in modern Python 3.12+.

1. The Four Pillars of OOP in Modern Python

Object-Oriented Programming (OOP) is a paradigm centered around objects that bundle state (data) and behavior (functions).



1. Encapsulation: Protecting Internal Invariants

Encapsulation prevents external code from corrupting an object's internal state. In Python, private attributes are denoted by a leading underscore `_` (convention) or double underscore `__` (name mangling):

2. Abstraction: Contract over Implementation

Hide complexity behind abstract base classes (`abc.ABC`) or structural protocols (`typing.Protocol`):

3. Inheritance vs Composition: Favor Composition

Inheritance creates a tight coupling ("is-a" relationship). In 95% of enterprise systems, **composition** ("has-a" relationship) yields significantly more maintainable architectures:

4. Polymorphism: Duck Typing & Dynamic Dispatch

Polymorphism allows different classes to respond to the same method call in their own specialized manner:

2. The SOLID Principles Masterclass

S: Single Responsibility → O: Open/Closed → L: Liskov Substitution → I: Interface Segregation → D: Dependency Inversion

S: Single Responsibility Principle (SRP)

"A class should have one, and only one, reason to change."

Anti-Pattern: The God Class

Production Refactor:

O: Open/Closed Principle (OCP)

"Software entities should be open for extension, but closed for modification."

Anti-Pattern: Conditionals on Types

Production Refactor (Strategy Pattern):



L: Liskov Substitution Principle (LSP)

"Subtypes must be substitutable for their base types without breaking program correctness."

Classic Violation: The Square-Rectangle Problem



Production Refactor:

Separate independent contracts or make shapes immutable value objects:



I: Interface Segregation Principle (ISP)

"Clients should not be forced to depend upon interfaces that they do not use."

Anti-Pattern: Fat Interfaces

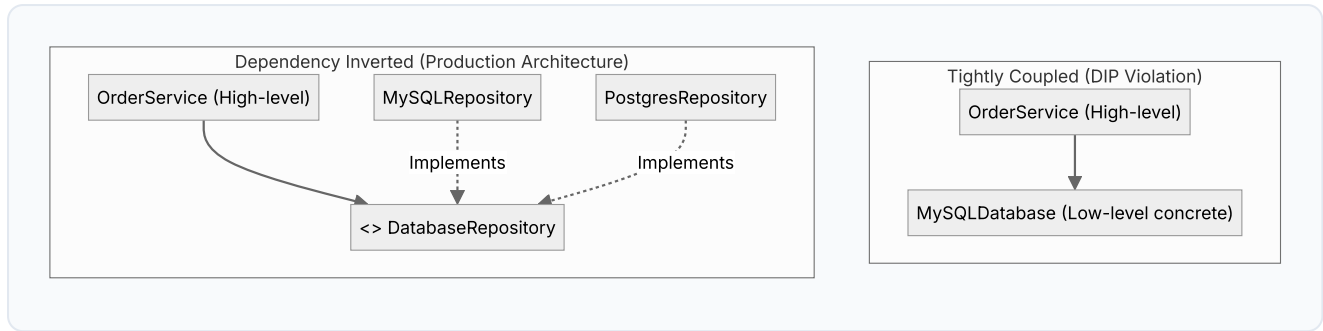


Production Refactor (Granular Interfaces):



D: Dependency Inversion Principle (DIP)

"High-level modules should not depend on low-level modules. Both should depend on abstractions."



Production Implementation:

Further Reading

- [Wikipedia: SOLID](#)
- [Refactoring Guru: SOLID principles](#)
- [Python abc module](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. State the Single Responsibility Principle with an example.

A class should have one reason to change. A `Report` class that both formats and emails reports changes for two reasons; split into `ReportFormatter` and `ReportMailer`.

▼ 2. What does the Liskov Substitution Principle forbid?

Subclasses that break the base class contract, e.g. a `Square` subclass of `Rectangle` that surprises callers by coupling width and height.

▼ 3. What does Dependency Inversion mean in practice?

High-level code depends on abstractions (interfaces/Protocols), and concrete implementations are injected, so they can be swapped and tested.

▼ 4. When does Open/Closed push you toward a pattern?

When new variants arrive frequently (payment methods, pricing rules): add new classes implementing an interface (Strategy) instead of editing an `if/else` chain.